

# Generative Modeling via SDEs

*From DDPM & SMLD to a Continuous-Time View*

$$d\mathbf{x} = f(\mathbf{x}, t) dt + g(t) d\mathbf{w}$$

**Sudarshan Babu**

*Slides created using Claude and Manus AI — each slide was prompted by the author.*

# Overview

1

## Unifying SMLD & DDPM

The corrupt-then-reverse paradigm — both are score-based models.

2

## Why an SDE Framework

Flexible sampling, better samplers & likelihoods, controllable generation.

3

## Forward & Reverse SDEs

The core objects: diffusing data into noise and reversing with the score.

4

## Deriving SDEs from Discrete Limits

VP from DDPM, VE from the SMLD noise ladder.

5

## The VP SDE

Variance-preserving diffusion — the continuous analog of DDPM.

6

## The VE SDE

Variance-exploding diffusion — the continuous analog of SMLD.

7

## Probability Flow ODE & the Unified Landscape

Deterministic sampling, exact likelihoods, one model many solvers.

8

## Experimental Results

CIFAR-10 — sampler comparison, exact likelihoods, and SOTA sample quality.

Central object: the score  $\nabla_x \log p_t(x)$  ties every method together. · Appendix: full VP  $\alpha/\sigma$  derivation.

PART 01

# Unifying SMLD & DDPM

*The corrupt-then-reverse paradigm*

SMLD · DDPM · The unifying insight · The score function

# The "Corrupt Then Reverse" Paradigm

*The unifying intuition behind SMLD, DDPM, and the SDE framework.*

## 1 · SEQUENTIALLY CORRUPT DATA

Slowly inject noise to transform the complex data distribution into a tractable prior (e.g., a Gaussian).

## 2 · LEARN TO REVERSE THE CORRUPTION

Train a model to undo the noise step-by-step, transforming the prior back into new data samples.

Corrupt forward into noise → learn the reverse → generate.

# SMLD: Score Matching with Langevin Dynamics

## TRAINING PHASE

**Perturb data with a discrete ladder of noise scales:**

$$\sigma_1 < \sigma_2 < \dots < \sigma_k$$

Estimate the score at each scale via denoising score matching:

$$s\theta(x, \sigma) \approx \nabla_x \log p\sigma(x)$$

## GENERATION PHASE

**Run Annealed Langevin Dynamics.**

Step through decreasing noise scales from noise to data:

$$\sigma_k \rightarrow \sigma_{k-1} \rightarrow \dots \rightarrow \sigma_1$$

At each scale, take Langevin MCMC steps guided by the estimated score toward high-density regions.

# DDPM: Denoising Diffusion Probabilistic Models

## TRAINING PHASE

Define a fixed Markov chain that gradually adds Gaussian noise over  $K$  steps.

Train a sequence of probabilistic reverse models step-by-step:

$$p_{\theta}(x_{t-1} | x_t)$$

Tractable because the reverse conditionals have a known Gaussian form.

## GENERATION PHASE

**Start from pure Gaussian noise:**

$$x_k \sim N(0, I)$$

Iteratively denoise the sample:

$$x_k \rightarrow x_{k-1} \rightarrow \dots \rightarrow x_0$$

Each step removes a small amount of noise using the learned reverse model.

# Unifying Viewpoint: Both Are Score-Based

## THE UNIFYING INSIGHT

In continuous state spaces, DDPM training implicitly computes scores at each noise scale.

## WHY THEY ARE DEEPLY RELATED

- ✓ Both estimate scores (gradients of log-density) of intermediate noisy distributions.
- ✓ Both use those estimated scores to guide a reverse denoising process.
- ✓ Apparent differences (Markov chain vs. Langevin dynamics) are merely implementation details of the same underlying principle.

# The Core Quantity: Scores Along the Corruption Path

$$\nabla_{\mathbf{x}} \log p_t(\mathbf{x})$$

*The score of the marginal distribution  $p_t(x)$  at time  $t$*

## DIRECTIONAL GUIDE

Points toward regions of higher data density  
— the exact direction needed to denoise.

## NORMALIZATION-FREE

Estimated without knowing the intractable  
normalizing constant of  $p_t(x)$ .

## TRACTABLE TRAINING

Denoising score matching is scalable and  
computationally efficient.

SMLD and DDPM are unified as score-based models — differing only in how they parameterize the noise process.



PART 02

# Why an SDE Framework?

*Enabling new samplers and capabilities*

Flexible sampling · Better samplers & likelihoods · Controllable generation

# Why an SDE Framework?

## PRIMARY GOAL

Extend score-based models by viewing them through stochastic differential equations.

### 01 · FLEXIBLE SAMPLING

Any general-purpose SDE solver can integrate the reverse process — expanding the design space beyond fixed discrete samplers.

### 02 · BETTER SAMPLERS & LIKELIHOODS

Unlocks Predictor–Corrector methods and the Probability Flow ODE, enabling exact likelihood computation.

### 03 · CONTROLLABLE GENERATION

Condition on information not available during training (e.g., inpainting) without retraining the model.

# Motivation (A): Flexible Sampling

*Once posed as an SDE, the reverse process can be integrated by any general-purpose solver.*

## THE SOLVER TOOLBOX

- Euler–Maruyama
- Milstein method
- Stochastic Runge–Kutta
- Predictor–Corrector methods

## EXPANDED CAPABILITIES

This expands the design space beyond fixed K-step discrete samplers.

Trade off speed vs. accuracy using adaptive step sizes and error-control mechanisms.

**Discrete DDPM and SMLD samplers become just two specific choices within a much larger space.**

# Motivation (B): Better Samplers & Exact Likelihoods

## PREDICTOR-CORRECTOR (PC)

Combines a numerical SDE solver step (predictor) with a score-based MCMC step (corrector).

### Benefit:

Improved sample quality at the same computational budget.

## PROBABILITY FLOW ODE

A deterministic ODE that shares the same marginal distributions as the stochastic SDE.

### Benefit:

Fast adaptive sampling using black-box ODE solvers.

**The Big Payoff — Exact Likelihood:** the ODE view computes the exact likelihood of data via the instantaneous change-of-variables formula — not directly available in the stochastic formulation.

# Motivation (C): Controllable Generation

*Modulate generation by conditioning on new information — without retraining the model.*

THE MECHANISM: BAYES' RULE ON SCORES

$$\nabla_x \log p_t(x \mid y) = \nabla_x \log p_t(x) + \nabla_x \log p_t(y \mid x)$$

*Conditional score = unconditional score (learned) + guidance term (estimated)*

APPLICATIONS — FROM ONE UNCONDITIONAL MODEL

- Image inpainting
- Colorization
- Class-conditioning
- Inverse problems (super-resolution)

# From Discrete Noise Ladders to a Continuum

## THE DISCRETE APPROACH

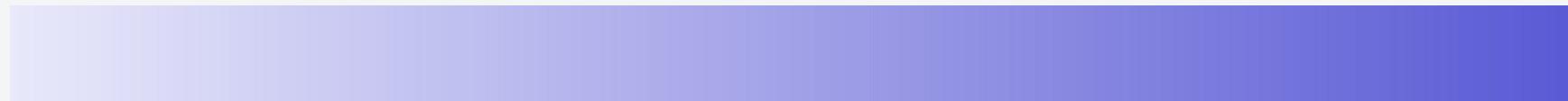
$(\sigma_1, \dots, \sigma_k)$  or  $(\beta_1, \dots, \beta_k)$

SMLD and DDPM rely on a finite set of noise levels or steps.

## THE CONTINUOUS VIEW

$\{p_t\}$  for  $t \in [0, 1]$

Consider a continuum of distributions indexed by continuous time  $t$ .



$t = 0$  · Data distribution  $p_{\text{data}}$

$t = 1$  · Prior  $N(0, I)$

PART 03

# Forward & Reverse SDEs

*The core objects of the framework*

Forward SDE · Reverse-time SDE · PC samplers · Probability Flow ODE

# Core Idea: The Forward SDE

*Diffusing data into noise.*

$$d\mathbf{x} = f(\mathbf{x}, t) dt + g(t) d\mathbf{w}$$

*$f(\mathbf{x}, t)$ : drift coefficient ·  $g(t)$ : diffusion coefficient ·  $d\mathbf{w}$ : Brownian motion*

## FIXED PROCESS

Contains no trainable parameters. Prescribed by design to match a specific noise schedule.

## DATA-INDEPENDENT

The process does not depend on the data distribution; it blindly adds noise.



# Core Idea: The Reverse-Time SDE

*For a forward SDE of the form on slide 16, the reverse is itself an SDE (Anderson, 1982).*

$$d\mathbf{x} = \left[ f(\mathbf{x}, t) - g(t)^2 \nabla_{\mathbf{x}} \log p_t(\mathbf{x}) \right] dt + g(t) d\bar{\mathbf{w}}$$

*Same  $f$  and  $g$  as the forward SDE (slide 16) — to reverse it, we only need the score of the marginals.*

## 1 Train Score Model

Train a time-dependent network via score matching:  $s_{\theta}(\mathbf{x}, t) \approx \nabla_{\mathbf{x}} \log p_t(\mathbf{x})$ .

## 2 Substitute

Plug the learned score  $s_{\theta}$  into the reverse-time SDE in place of the true score.

## 3 Integrate

Solve the SDE numerically from  $t=1$  (noise) to  $t=0$  (data) to generate samples.

**Derivation:** the reverse-time SDE is proved in Anderson (1982); see also Song et al. (2021), Appendix A.

# Contribution #1: A Sampling Toolbox (PC Samplers)

## THE PREDICTOR

### Numerical SDE step.

Uses a standard solver (e.g., Euler–Maruyama) to estimate the sample at the next time step.

## THE CORRECTOR

### Score-based MCMC.

Runs Langevin dynamics or HMC to correct the marginal distribution and refine the sample.

## THE RESULT

**Predictor + Corrector  $\Rightarrow$  higher sample quality at the same compute budget.**

# Contribution #2: The Probability Flow ODE

*For the same forward SDE on slide 16: a deterministic ODE with the same marginals as the reverse SDE.*

$$d\mathbf{x} = \left[ f(\mathbf{x}, t) - \frac{1}{2} g(t)^2 \nabla_{\mathbf{x}} \log p_t(\mathbf{x}) \right] dt$$

*Same  $f$  and  $g$  as the forward SDE (slide 16); note the factor of  $\frac{1}{2}$  on the score vs. the reverse SDE.*

## FAST ADAPTIVE SAMPLING

Use black-box ODE solvers (e.g., Dormand–Prince) with adaptive step sizes.

## FLEXIBLE LATENT SPACE

Deterministic encoding of data to noise — interpolation and manipulation in latent space.

## EXACT LIKELIHOODS

Via the instantaneous change-of-variables formula, avoiding ELBO approximations.

**Derivation:** the probability-flow ODE is derived in Song et al. (2021), Appendix D.1, from the same forward SDE.

# Why It Works: Stay on the Forward Marginals

*The network gives the score at a point — reliable only where the distribution is correct.*



## IT KNOWS THE SCORE, NOT THE DISTRIBUTION

At a point  $x$  and time  $t$  the network returns the score — the local direction toward higher density. It was trained on samples from  $p_t$ , so it is only accurate where  $p_t$  has mass.

## CORRECT IN → CORRECT OUT

If the population at time  $t$  is distributed as  $p_t$ , stepping every point along the score makes the next population correct too. Start from the right prior and correctness propagates down to the data.

## OFF-DISTRIBUTION → IT BREAKS

If points drift off  $p_t$ , the network is queried out-of-distribution and returns wrong scores, so errors compound. The one thing that matters: keep each intermediate marginal equal to the forward  $p_t$ .

**At each time, match the reverse process's  $p_t$  to the forward process's  $p_t$ , so the network produces an accurate score.**  
Hence the probability-flow ODE (previous slide) — which maintains exactly these marginals — yields valid deterministic sampling.

# Contribution #3: A Unified Framework

*SMLD and DDPM as discretizations of two continuous-time SDEs.*

## VE SDE · VARIANCE EXPLODING

**SMLD** → the variance of the process grows without bound as  $t \rightarrow \infty$ .

## VP SDE · VARIANCE PRESERVING

**DDPM** → the variance is bounded (typically at 1) throughout the process.

**Key implication:** the familiar discrete algorithms are not fundamentally different — they are two specific discretizations of two continuous-time SDEs within the same framework.

# Roadmap: VP vs. VE and Next Steps

## 1 Choose Forward SDE

Select a noise process (e.g., VE, VP) to transform data into the prior.

## 2 Learn Score

Train  $s\theta(x,t) \approx \nabla \log p_t(x)$  via score matching.

## 3 Sample

Integrate the reverse SDE or the probability-flow ODE.

### VP SDE · VARIANCE PRESERVING

**A forward-SDE diffusion (noising) process.**

Shrinks the signal as it adds noise, so the total variance stays bounded ( $\approx 1$ ).

→ **The continuous analog of DDPM.**

### VE SDE · VARIANCE EXPLODING

**A forward-SDE diffusion (noising) process.**

Keeps adding ever-larger noise with no signal decay, so the variance grows without bound.

→ **The continuous analog of SMLD.**

**VP and VE are the two canonical SDE diffusion methods — forward processes that turn data into noise, differing only in how the variance behaves.**

PART 04

# Deriving SDEs from Discrete Limits

*VP from DDPM · VE from the SMLD ladder*

Taylor expansion · Brownian increments · Forward & reverse SDEs

# Goal: Derive Forward SDEs from Discrete Processes

## DDPM FORWARD CHAIN → VP SDE

The variance-preserving limit of the discrete Gaussian chain.

## SMLD NOISE LADDER → VE SDE

The variance-exploding limit of the discrete noise ladder.

### THE STRATEGY

- 1 Rescale noise to make each step infinitesimal.
- 2 Write the increment  $x(t+\Delta t) - x(t)$ .
- 3 Take the limit  $\Delta t \rightarrow 0$ .
- 4 Identify the drift and diffusion terms.



# VP: Start from the DDPM Forward Chain

$$\mathbf{x}_i = \sqrt{1 - \beta_i} \mathbf{x}_{i-1} + \sqrt{\beta_i} \mathbf{z}_{i-1}$$

where  $\mathbf{z}_{i-1} \sim N(0, I)$

## SCALING FOR $N \rightarrow \infty$

**Define  $\bar{\beta}_i = N \cdot \beta_i$ .**

This keeps the total noise variance over  $[0,1]$  finite as the number of steps increases.

## TIME VARIABLES

**Let  $t = i/N$  and  $\Delta t = 1/N$ .**

Thus  $\beta_i = \bar{\beta}(t) \cdot \Delta t$ .

## Rewrite in Continuous-Time Form

$$\mathbf{x}(t + \Delta t) = \sqrt{1 - \beta(t + \Delta t)\Delta t} \mathbf{x}(t) + \sqrt{\beta(t + \Delta t)\Delta t} \mathbf{z}(t)$$

Now we expand this for small  $\Delta t$  ...

# Taylor Expand the Drift Term

## APPROXIMATION

**Approximation:**  $\sqrt{1 - \varepsilon} \approx 1 - \frac{1}{2}\varepsilon$

**Apply to drift:**  $\sqrt{1 - \beta(t)\Delta t} \approx 1 - \frac{1}{2}\beta(t)\Delta t$

**Substitute back:**  $\mathbf{x}(t+\Delta t) \approx (1 - \frac{1}{2}\beta(t)\Delta t) \cdot \mathbf{x}(t) + \sqrt{\beta(t)\Delta t} \cdot \mathbf{z}(t)$

$$\mathbf{x}(t + \Delta t) - \mathbf{x}(t) \approx -\frac{1}{2}\beta(t)\mathbf{x}(t)\Delta t + \sqrt{\beta(t)\Delta t} \mathbf{z}(t)$$

# Identify the Brownian Increment

$$\sqrt{\Delta t} \mathbf{z}(t) \equiv d\mathbf{w}_t \quad (\text{since } \mathbf{z}(t) \sim N(0, I))$$

## RESULTING VP FORWARD SDE

$$d\mathbf{x} = -\frac{1}{2}\beta(t)\mathbf{x} dt + \sqrt{\beta(t)} d\mathbf{w}_t$$

*Taking the limit  $\Delta t \rightarrow 0$  yields this linear SDE.*

# VE: Start from Incremental Variance

## SMLD GOAL

$$\mathbf{x}_k = \mathbf{x}_0 + \sigma_k \boldsymbol{\varepsilon}$$

## MARKOV CHAIN CONSTRUCTION

To satisfy the marginals, the incremental step must be:

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \sqrt{\sigma_{k+1}^2 - \sigma_k^2} \mathbf{z}_k$$

$$\mathbf{z}_k \sim N(0, I)$$

# Take the Continuous-Time Limit for VE

## DERIVATION

1 · **Variance difference:**  $\sigma^2(t+\Delta t) - \sigma^2(t) \approx d/dt[\sigma^2(t)] \cdot \Delta t$

2 · **Substitute into increment:**  $x(t+\Delta t) - x(t) = \sqrt{(d/dt[\sigma^2(t)] \cdot \Delta t)} \cdot z(t)$

3 · **Identify Brownian motion:**  $\sqrt{\Delta t} \cdot z(t) = dw_t$

## RESULTING VE FORWARD SDE

$$dx = \sqrt{\frac{d}{dt}\sigma^2(t)} dw$$

# Summary of Forward SDEs

VP (DDPM)

$$d\mathbf{x} = -\frac{1}{2}\beta(t)\mathbf{x} dt + \sqrt{\beta(t)} d\mathbf{w}_t$$

VE (SMLD)

$$d\mathbf{x} = \sqrt{\frac{d}{dt}\sigma^2(t)} d\mathbf{w}_t$$

Both are linear Itô diffusions. The key difference is the presence of the drift term in VP.

# Reverse-Time SDE (Anderson, 1982)

FORWARD

$$d\mathbf{x} = f(\mathbf{x}, t) dt + g(t) d\mathbf{w}_t$$

REVERSE

$$d\mathbf{x} = \left[ f(\mathbf{x}, t) - g^2(t) \nabla \log p_t(\mathbf{x}) \right] dt + g(t) d\bar{\mathbf{w}}_t$$

*Reversal introduces an additional drift term,  $-g^2(t) \cdot \text{score}$ . This is why estimating the score lets us reverse the diffusion.*



# Reverse SDE for VP

$$f(\mathbf{x}, t) = -\frac{1}{2}\beta(t)\mathbf{x} \quad g(t) = \sqrt{\beta(t)}$$

## RESULTING REVERSE VP SDE

$$d\mathbf{x} = \left[ -\frac{1}{2}\beta(t)\mathbf{x} - \beta(t)s_{\theta}(\mathbf{x}, t) \right] dt + \sqrt{\beta(t)} d\bar{\mathbf{w}}$$

Replace the unknown score  $\nabla \log p_t(\mathbf{x})$  with the trained network  $s_{\theta}(\mathbf{x}, t)$ .

# Reverse SDE for VE

$$f(\mathbf{x}, t) = 0 \quad g(t) = \sqrt{\frac{d}{dt}[\sigma^2(t)]}$$

## RESULTING REVERSE VE SDE

$$d\mathbf{x} = \left[ -g^2(t) s_{\theta}(\mathbf{x}, t) \right] dt + g(t) d\bar{\mathbf{w}}$$

**Notice:** no anchor drift term ( $-\frac{1}{2}\beta x$ ) here. The reverse process is driven purely by the score.

# What We Achieved

## FORWARD SDES

- VP derived from the DDPM limit
- VE derived from the noise-ladder limit

## REVERSE SDES

- Obtained directly via Anderson's theorem
- No posterior derivations required
- No discrete reverse conditionals needed

Sampling now reduces to a standard numerical problem: solving the reverse SDE.

PART 05

# The VP SDE

*Variance-preserving diffusion — the continuous analog of DDPM*

Bounded variance · Closed-form marginals · Reverse SDE · DDPM samplers

# VP SDE: Variance-Preserving Diffusion

## CORE DEFINITION

Variance stays bounded as time increases, preventing the signal scale from blowing up.

## THE MECHANISM

Slowly attenuate the signal while injecting noise, so the total variance stays controlled (often  $\approx 1$ ).

## THE CONNECTION

The direct continuous-time analog of DDPM-style discrete noising schedules.

# VP Forward SDE Definition

[Derived → slide 28](#)

$$d\mathbf{x} = -\frac{1}{2}\beta(t)\mathbf{x} dt + \sqrt{\beta(t)} d\mathbf{w}_t$$

**DRIFT TERM** ·  $-\frac{1}{2}\beta(t)\mathbf{x}$

Pulls the data toward 0, attenuating the signal to keep variance controlled.

**DIFFUSION TERM** ·  $\sqrt{\beta(t)} d\mathbf{w}_t$

Injects noise at a rate set by the nonnegative schedule  $\beta(t)$ .

# VP: From SDE to Closed-Form Solution

[Full derivation →](#)

The VP SDE is linear, so its transition kernel  $p_t(x | x_0)$  is Gaussian — fixed entirely by a mean and a variance.

## THE MEAN · SIGNAL DECAY

Drop the noise: the drift  $-\frac{1}{2}\beta(t)x$  only rescales the signal.

Its integrating factor gives the signal's decay  $\alpha(t)$ :

$$\alpha(t) = \exp\left(-\frac{1}{2} \int_0^t \beta(s) ds\right)$$

## THE VARIANCE · NOISE BUILD-UP

Injected noise builds up as  $d\Sigma/dt = -\beta(t)\Sigma + \beta(t)$ , from  $\Sigma(0)=0$ .

Solving this linear ODE gives  $\sigma^2(t)$ :

$$\sigma^2(t) = 1 - \alpha^2(t)$$

## THE SAME PROCESS, IN CLOSED FORM

$$\mathbf{x}(t) = \alpha(t) \mathbf{x}(0) + \sigma(t) \mathbf{z}, \quad \mathbf{z} \sim N(0, I)$$

Exactly the solution of the VP forward SDE — the differential equation and this closed form describe the same process, so you can jump to any time  $t$  in one step.

# VP: Closed-Form Marginals

$$\mathbf{x}(t) = \alpha(t)\mathbf{x}(0) + \sigma(t)\mathbf{z}, \quad \mathbf{z} \sim N(0, I)$$

$$\alpha(t) = \exp(-\tfrac{1}{2}\int\beta(s)ds) \quad \sigma^2(t) = 1 - \alpha^2(t)$$

**Key property — Bounded Variance:** because  $\alpha^2(t) + \sigma^2(t) = 1$ , the total variance of  $\mathbf{x}(t)$  stays controlled (typically  $\approx 1$ ) throughout the process, preventing numerical instability.



# VP: Score-Model Target

$$s_{\theta}(\mathbf{x}, t) \approx \nabla_{\mathbf{x}} \log p_t(\mathbf{x})$$

## DIRECT SCORE

Train the network to output the score vector directly.

## NOISE PREDICTION

Predict the noise  $\epsilon_{\theta}$  and convert:

$$\mathbf{s}_{\theta} = -\epsilon_{\theta} / \sigma(t)$$

DDPM's noise-prediction is essentially a reparameterization of the score in this Gaussian setting.

# VP: Reverse-Time SDE

[Derived → slide 33](#)

GENERAL REVERSE TEMPLATE

$$d\mathbf{x} = \left[ f(\mathbf{x}, t) - g^2(t) \nabla \log p_t(\mathbf{x}) \right] dt + g(t) d\bar{\mathbf{w}}$$

For VP:  $f(\mathbf{x}, t) = -\frac{1}{2}\beta(t)\mathbf{x}$       $g(t) = \sqrt{\beta(t)}$

RESULTING REVERSE VP SDE

$$d\mathbf{x} = \left[ -\frac{1}{2}\beta(t)\mathbf{x} - \beta(t)s_\theta(\mathbf{x}, t) \right] dt + \sqrt{\beta(t)} d\bar{\mathbf{w}}$$

# VP: Discretizations & DDPM

Discretize the VP reverse process with a specific timestep rule, and you recover DDPM-like samplers.

## ANCESTRAL SAMPLING

Corresponds to using the reverse conditional Gaussian  $p(x_{t-1}|x_t)$  at each step — the standard DDPM sampler.

## SOLVER FLEXIBILITY

Different discretizations (Euler–Maruyama, higher-order methods) yield different algorithms from the same model.

# VP: Sampling in Practice

## SCHEDULE SELECTION

Performance depends heavily on a good  $\beta(t)$  schedule. Cosine schedules often outperform linear ones by preserving signal longer.

## DISCRETIZATION CHOICE

Deterministic variants like DDIM are possible via the ODE view, offering faster sampling.

Empirically, VP/DDPM gives strong perceptual quality with standard architectures and samplers.

# VP SDE: Summary



## Forward Process

Bounded variance, linear drift to 0, plus diffusion.



## Training

Estimate the time-dependent score (or noise).



## Sampling

Integrate the reverse SDE (stochastic) or the probability-flow ODE (deterministic).



## Connection

Discretizations connect directly to the DDPM family.

PART 06

# The VE SDE

*Variance-exploding diffusion — the continuous analog of SMLD*

Exploding variance · DSM · Reverse SDE · Predictor–Corrector

# VE SDE: Variance-Exploding Diffusion

## CORE DEFINITION

Noise scale grows dramatically with time — the variance 'explodes'.

## THE MECHANISM

Instead of damping the signal, simply add more and more noise until the distribution is effectively pure Gaussian.

## THE CONNECTION

Matches the noise-level ladder intuition in SMLD (Song & Ermon, 2019).

# VE Forward SDE Definition

[Derived → slide 30](#)

$$d\mathbf{x} = g(t) d\mathbf{w}_t$$

## PURE DIFFUSION

No drift term. The process is driven purely by Brownian motion scaled by  $g(t)$ .

## NOISE SCALE

$g(t)$  is chosen so the noise standard deviation increases from  $\sigma_{\min}$  to  $\sigma_{\max}$ .



# VE: Closed-Form Marginals

$$\mathbf{x}(t) = \mathbf{x}(0) + \sigma(t)\mathbf{z}, \quad \mathbf{z} \sim N(0, I)$$

*Pure diffusion: data convolved with Gaussian noise*

**Connection to SMLD:** this is exactly the object SMLD trains on — scores of Gaussian-smoothed densities across a range of noise levels  $\sigma(t)$ .

# VE: Score Model & DSM

$$s_{\theta}(\mathbf{x}, t) \approx \nabla_{\mathbf{x}} \log p_t(\mathbf{x})$$

## CONTINUUM OF SIGMAS

Corresponds to Denoising Score Matching (DSM) across a continuous range of noise scales.

## SMLD CONNECTION

SMLD's 'discrete sigmas + Langevin per sigma' is simply a discretized version of this continuous-time story.

We learn the gradient of the log-density for every noise level in the explosion process.

# VE: Reverse-Time SDE

[Derived → slide 34](#)

Forward parameters:  $f(\mathbf{x}, t) = 0$       $g(t) = \text{chosen schedule}$

## RESULTING REVERSE VE SDE

$$d\mathbf{x} = \left[ -g^2(t) s_{\theta}(\mathbf{x}, t) \right] dt + g(t) d\bar{\mathbf{w}}$$

**Compare to VP:** no anchor-drift term ( $-\frac{1}{2}\beta\mathbf{x}$ ) here. The drift is purely score-driven.

# VE: Discretizations & SMLD

The SMLD sampler is essentially a specific Predictor–Corrector discretization of the VE SDE.

## PREDICTOR STEP

A standard numerical SDE solver step (e.g., Euler–Maruyama) moves the sample backward in time.

## CORRECTOR STEP

Langevin MCMC steps (at fixed time  $t$ ) correct the sample to match the marginal  $p_t(x)$ .

Result: recovering SMLD-style annealed Langevin dynamics.

# VE: Practical Sampling — Why PC Shines

## PREDICTOR

**Step forward** — makes progress along time, solving the reverse SDE.

## CORRECTOR (LANGEVIN)

**Re-equilibrate** — pulls the sample toward the marginal  $p_t(x)$  at the current noise level.

**Why it matters:** with large noise scales the score landscape changes rapidly; the corrector stabilizes the trajectory by pulling samples back to the high-probability manifold.

# VE SDE: Summary



## Forward Process

Pure diffusion; noise std increases from  $\sigma_{\min}$   $\rightarrow$   $\sigma_{\max}$ .



## Training

Scores of Gaussian-smoothed densities (DSM).



## Sampling

Reverse SDE, often with Predictor–Corrector steps.



## Connection

Discretizations connect directly to the SMLD family.

PART 07

# Probability Flow ODE & the Unified Landscape

*Deterministic sampling, exact likelihoods, one model — many solvers*

Probability Flow ODE · Matching marginals · SDE vs ODE · The bridge

# Probability Flow: Why?

With the same learned score  $s_\theta(x,t)$ , we can define deterministic dynamics.

## DETERMINISM

Avoids sampling noise. Given an initial noise vector, the output is fixed.

## ADAPTIVE SOLVERS

Enables black-box ODE solvers (RK45, etc.) for error control and speed.

## EXACT LIKELIHOODS

A clean route to log-likelihoods via the change-of-variables formula.



# Probability Flow ODE Definition

[Introduced → slide 19](#)

GIVEN FORWARD SDE

$$d\mathbf{x} = f(\mathbf{x}, t) dt + g(t) d\mathbf{w}_t$$

**PROBABILITY FLOW ODE**

$$d\mathbf{x} = \left[ f(\mathbf{x}, t) - \frac{1}{2} g^2(t) \nabla \log p_t(\mathbf{x}) \right] dt$$

*In practice, replace the true score  $\nabla \log p_t(\mathbf{x})$  with the learned model  $s_\theta(\mathbf{x}, t)$ .*

# ODE Property: Matching Marginals

## CRUCIAL GUARANTEE

The Probability Flow ODE shares the exact same marginal distributions  $p_t(x)$  as the corresponding stochastic SDE.

## THE IMPLICATION

Integrating this ODE from 'noise time' back to 'data time' yields samples from the same target distribution (in the idealized limit).

# VP Probability Flow ODE

[General form → slide 57](#)

$$\mathbf{f} = -\frac{1}{2}\beta(t)\mathbf{x} \quad \mathbf{g}^2 = \beta(t)$$

## VP PROBABILITY FLOW ODE

$$d\mathbf{x} = \left[ -\frac{1}{2}\beta(t)\mathbf{x} - \frac{1}{2}\beta(t)s_{\theta}(\mathbf{x}, t) \right] dt$$

Deterministic alternative to DDPM/VP reverse-SDE sampling. Note the  $\frac{1}{2}$  on the score term.

# VE Probability Flow ODE

[General form → slide 57](#)

$$f = 0 \quad g^2 = g^2(t)$$

## VE PROBABILITY FLOW ODE

$$d\mathbf{x} = \left[ -\frac{1}{2}g^2(t)s_\theta(\mathbf{x}, t) \right] dt$$

A deterministic flow guided purely by the score — simple, drift-free structure.

# ODE Payoff: Adaptive Sampling

## ERROR CONTROL

ODE solvers automatically adjust step sizes based on local error estimates.

## DYNAMIC ALLOCATION

Spend more steps where dynamics are stiff (curved) and fewer where the flow is straight.

**Number of Function Evaluations (NFE) becomes a runtime knob — not a fixed hyperparameter  $K$ .**

# ODE Payoff: Exact Likelihoods

$$\log p(\mathbf{x}(0)) = \log p(\mathbf{x}(T)) - \int_0^T \text{tr}(\nabla_{\mathbf{x}} f_{\text{ode}}) dt$$

## CONTINUOUS NORMALIZING FLOW

The ODE is a continuous invertible mapping — compute log-likelihood via the instantaneous change-of-variables formula.

## HUTCHINSON'S TRICK

The trace term is estimated stochastically to avoid the full Jacobian, scaling to high dimensions.

# SDE vs. ODE: When to Use Which

## REVERSE SDE

- ✓ Stochasticity can correct errors
- ✓ Robust to score inaccuracies
- ✓ Supports Predictor–Corrector sampling

## PROBABILITY FLOW ODE

- ✓ Deterministic (fixed latent map)
- ✓ Adaptive solvers (faster)
- ✓ Exact likelihood computation
- ✓ Latent-space manipulation

Crucially, both methods use the exact same trained score model  $s_\theta(x,t)$ .

# Bridge: The Unified Landscape

## VP SDE DISCRETIZATIONS

→ DDPM family

## VE SDE DISCRETIZATIONS

→ SMLD family

SDE framework: one score model + many solvers / discretizations.



# Bridge: Dimensions of Comparison

## SCHEDULES

$\beta(t)$  for VP  
 $\sigma(t)$  /  $g(t)$  for VE

## SAMPLERS

Ancestral (DDPM)  
DDIM (deterministic)  
Euler–Maruyama  
Predictor–Corrector  
ODE (adaptive)

## TRADE-OFFS

Sample quality  
Speed (NFE)  
Likelihood  
Controllability

One framework, many knobs — quality, speed, likelihood, and control.

PART 08

# Experimental Results

*Song et al. (2021) — what the SDE framework delivers on CIFAR-10*

Samplers · Likelihoods · Sample quality

# Results at a Glance

*CIFAR-10, unconditional generation — and the first high-fidelity score-based  $1024^2$  samples.*

**2.20**

**FID ↓**

New SOTA on CIFAR-10 (NCSN++  
cont., deep VE).

**9.89**

**Inception ↑**

Highest IS — beats the best GANs,  
no labels.

**2.99**

**bits/dim ↓**

Record likelihood via the probability-  
flow ODE (sub-VP).

**$1024^2$**

**first samples**

First high-fidelity images from a score-  
based model.

**One framework — best-in-class samples (VE) and record likelihoods (sub-VP) — from the same score model.**

# Table 1 — Reverse-Time SDE Solvers (FID)

|                    | Variance Exploding SDE (SMLD) |       |       |        | Variance Preserving SDE (DDPM) |       |       |        |
|--------------------|-------------------------------|-------|-------|--------|--------------------------------|-------|-------|--------|
| FID · Sampler      | P1000                         | P2000 | C2000 | PC1000 | P1000                          | P2000 | C2000 | PC1000 |
| ancestral sampling | 4.98                          | 4.88  | —     | 3.62   | 3.24                           | 3.24  | —     | 3.21   |
| reverse diffusion  | 4.79                          | 4.74  | 20.43 | 3.60   | 3.21                           | 3.19  | 19.06 | 3.18   |
| probability flow   | 15.41                         | 10.54 | —     | 3.51   | 3.59                           | 3.23  | —     | 3.06   |

**Why these numbers:** Predictor+Corrector (PC1000) is best everywhere — VP probability-flow PC hits 3.06. Corrector-only (C2000) collapses ( $\approx 19$ – $20$ ) at equal compute: pure MCMC mixes too slowly. Reverse-diffusion beats ancestral sampling, and for VE, predictor-only struggles (15.41) until a corrector rescues it.

## Table 2 — Likelihoods (NLL) & FID via the ODE

| Model                              | NLL ↓         | FID ↓       |
|------------------------------------|---------------|-------------|
| RealNVP                            | 3.49          | —           |
| Glow                               | 3.35          | —           |
| Flow++                             | 3.29          | —           |
| Residual Flow                      | 3.28          | 46.37       |
| DDPM (L_simple)                    | $\leq 3.75^*$ | 3.17        |
| DDPM (ours)                        | 3.28          | 3.37        |
| DDPM cont. (VP)                    | 3.21          | 3.69        |
| DDPM cont. (sub-VP)                | 3.05          | 3.56        |
| DDPM++ cont. (VP)                  | 3.16          | 3.93        |
| DDPM++ cont. (sub-VP)              | 3.02          | 3.16        |
| DDPM++ cont. (deep, VP)            | 3.13          | 3.08        |
| <b>DDPM++ cont. (deep, sub-VP)</b> | <b>2.99</b>   | <b>2.92</b> |

### WHY THE LIKELIHOODS IMPROVE

**Exact, not ELBO** — the ODE gives exact log-likelihood, so the same DDPM beats its own ELBO bound ( $3.28 < 3.70$ ).

**Continuous objective helps** — training with the continuous loss (DDPM cont.) lowers bits/dim further.

**sub-VP wins on likelihood** — sub-VP always edges out VP, because its variance never exceeds the VP path.

**Record** — DDPM++ cont. (deep, sub-VP) sets 2.99 bits/dim — without any maximum-likelihood training.

## Table 3 — CIFAR-10 Sample Quality (FID & IS)

| Model                          | FID ↓       | IS ↑        |
|--------------------------------|-------------|-------------|
| Conditional                    |             |             |
| BigGAN                         | 14.73       | 9.22        |
| StyleGAN2-ADA                  | 2.42        | 10.14       |
| Unconditional                  |             |             |
| StyleGAN2-ADA                  | 2.92        | 9.83        |
| NCSN                           | 25.32       | 8.87        |
| NCSNv2                         | 10.87       | 8.40        |
| DDPM                           | 3.17        | 9.46        |
| DDPM++ cont. (VP)              | 2.55        | 9.58        |
| DDPM++ cont. (deep, VP)        | 2.41        | 9.68        |
| NCSN++                         | 2.45        | 9.73        |
| NCSN++ cont. (VE)              | 2.38        | 9.83        |
| <b>NCSN++ cont. (deep, VE)</b> | <b>2.20</b> | <b>9.89</b> |

### WHY VE WINS ON SAMPLES

**VE for fidelity** — NCSN++ cont. (deep, VE) reaches FID 2.20 / IS 9.89 — the best in the table.

**Beats the GANs** — it surpasses StyleGAN2-ADA (2.92) unconditionally, and even the best conditional model — with no labels.

**Depth + continuous training** — deeper NCSN++/DDPM++ and the continuous objective both push FID down.

**Mirror image of Table 2** — VE is best for samples while sub-VP is best for likelihood — same score model, different solver/SDE.

APPENDIX

# Derivation of the VP Marginals

*Why  $\alpha(t)$  and  $\sigma(t)$  take their closed forms*

Linear SDE · Integrating factor · Itô isometry

# Appendix — Deriving the Mean $\alpha(t)$

[Back to slide 39](#)

The VP SDE is linear, so  $x(t) \mid x(0)$  is Gaussian. Solve it explicitly — first the mean.

1 · Linear SDE

$$d\mathbf{x} = -\frac{1}{2}\beta(t)\mathbf{x}dt + \sqrt{\beta(t)}d\mathbf{w}$$

2 · Integrating factor

$$\mu(t) = \exp\left(\frac{1}{2}\int_0^t \beta(s)ds\right) \Rightarrow d(\mu\mathbf{x}) = \mu\sqrt{\beta}d\mathbf{w}$$

3 · Integrate  $0 \rightarrow t$

$$\mu(t)\mathbf{x}(t) = \mathbf{x}(0) + \int_0^t \mu(s)\sqrt{\beta(s)}d\mathbf{w}(s)$$

4 · Solve for  $\mathbf{x}(t)$

$$\mathbf{x}(t) = \alpha(t)\mathbf{x}(0) + \alpha(t)\int_0^t \mu(s)\sqrt{\beta(s)}d\mathbf{w}(s), \quad \alpha = \frac{1}{\mu}$$

5 · Take expectation

$$\mathbb{E}[\mathbf{x}(t) \mid \mathbf{x}(0)] = \alpha(t)\mathbf{x}(0), \quad \alpha(t) = \exp\left(-\frac{1}{2}\int_0^t \beta(s)ds\right)$$

The Itô integral has zero mean, so the conditional mean is simply  $\alpha(t)\cdot x(0)$ . Continues on the next slide →



# Appendix — Deriving the Variance $\sigma^2(t)$

[Back to slide 39](#)

The variance of the same solution, from the stochastic integral via Itô isometry.

1 · Random part

$$\mathbf{x}(t) - \alpha(t)\mathbf{x}(0) = \alpha(t) \int_0^t \mu(s) \sqrt{\beta(s)} d\mathbf{w}(s)$$

2 · Itô isometry

$$\text{Var}[\mathbf{x}(t)] = \alpha^2(t) \int_0^t \mu^2(s) \beta(s) ds = \alpha^2(t) \int_0^t e^{\int_0^s \beta(u) du} \beta(s) ds$$

3 · Perfect derivative

$$e^{\int_0^s \beta} \beta(s) = \frac{d}{ds} e^{\int_0^s \beta} \Rightarrow \int_0^t (\cdot) ds = e^{\int_0^t \beta} - 1 = \alpha^{-2}(t) - 1$$

4 · Collect

$$\sigma^2(t) = \alpha^2(t) (\alpha^{-2}(t) - 1) = 1 - \alpha^2(t)$$

**Variance preserving:** with normalized data,  $\alpha^2(t) + \sigma^2(t) = 1$ , so the marginal variance of  $\mathbf{x}(t)$  stays  $\approx 1$  throughout the process — the property that names the VP SDE.

# References

[1] **Song & Ermon (2019)**. *Generative Modeling by Estimating Gradients of the Data Distribution*. NeurIPS.

[2] **Sohl-Dickstein et al. (2015)**. *Deep Unsupervised Learning using Nonequilibrium Thermodynamics*. ICML.

[3] **Ho, Jain & Abbeel (2020)**. *Denoising Diffusion Probabilistic Models*. NeurIPS.

[4] **Anderson (1982)**. *Reverse-Time Diffusion Equation Models*. Stoch. Proc. Appl.

[5] **Hutchinson (1990)**. *A Stochastic Estimator of the Trace of the Influence Matrix*. Commun. Stat.

[6] **Song et al. (2021)**. *Score-Based Generative Modeling through SDEs*. ICLR.

[7] **Karras et al. (2020)**. *Training GANs with Limited Data (StyleGAN2-ADA)*. NeurIPS.

[8] **Brock et al. (2018)**. *Large Scale GAN Training (BigGAN)*. ICLR.